

Reflection: Growth Across Two AI Product Studios

Boyi Ke

Indiana Wesleyan University — AI Product Studio

October 2026

This studio asked me to ship twice. CashFlow Guardian AI (github.com/boyi17946/cashflow-guardian-ai) is a 13-week liquidity radar for a Northstar-class fabricator. CNH Plant Desk (github.com/boyi17946/cnh-plant-desk) is a tablet for Fargo combine final and Racine dealer-prep, where downtime still travels by radio and paper tag. Those two products changed how I define “done,” how I use AI, and what I refuse to fake when evidence is missing.

Growth through those projects

On the first app I still thought like a coursework builder. I split the stack because pandas felt native in Python and the UI felt native in Next.js. That produced a real ensemble—seasonal naive, exponential smoothing, and residual quantiles into P10/P50/P90—and a real tenant story with Maya and Luis. It also produced two terminals, CORS between ports 3000 and 8000, and an App Runner file that hosts the frontend while the forecast API stays local. I learned that cash numbers must never look like a bank wire, and that a clever architecture that cannot boot as one service is a classroom demo, not an operating desk.

The second app is where growth showed up as subtraction. I collapsed UI and APIs into one Next.js process, a JSON plant store, and a catalog ranker grounded in TSB steps and BOM SKUs. Live models may rewrite ranking copy; they may not invent procedures or parts. I packaged App Runner config before the GitHub push and published a second public repository instead of bolting plant code onto the cash repo. I still do not have an App Runner URL: AWS returned *NoCredentials* after an Identity Center device code expired. Growth included saying that out loud instead of inventing a live host for the assignment. I also grew as a tester. The plant desk looked finished until the browser proved it was not: a skeleton that never hydrated, dead clicks from a 403 on `/_next`, and a start command that fails with standalone output. Clicking the shift board taught me more than reading my own README.

How AI assistance changed my development approach

I used to treat a language model as a faster search engine: generate a file, paste, hope. That fails when a wrong P50 can scare a payroll, or a hallucinated SKU can send a tech to the wrong crib bin. The useful pattern became constraint-first prompting. For cash: never move money; show bands, not “the number.” For the plant: rewrite copy only; rank from plant knowledge; do not invent TSB steps. Official scaffolds beat hand-rolled config. A browser pass caught bugs I would have shipped. Once GitHub login worked, the second repo was one command. When App Runner was not created, I kept the honest artifacts: GitHub, local port 8080, and a labeled AWS sign-in screenshot.

The loop inverted. I still ask the model to draft. I hold the merge criteria: a health probe, empty and error states, real plant copy, and a boot path that needs no secrets. AI types quickly. I own the retrieval lock and the deploy contract.

Most valuable lesson learned

The operating system of the product matters more than the cleverness of the model. Between the two apps I did not “use more AI.” I used a stricter one. CashFlow Guardian’s intelligence is classical time series so a chatbot cannot invent runway. Plant Desk’s intelligence is a catalog ranker so a language model cannot invent a wrench sequence. Deployment belongs in that operating system: one process, one port, one health path. Honesty belongs too. A fabricated App Runner URL would have been the easiest slide in the deck and the least true thing I could have submitted. Feedback must also change a control, not a slogan. “Don’t make me trust one number” kept the fan chart. “I need a paper trail” kept the audit log. “Techs want ranked modes in a sheet, not a binder” became the diagnosis panel. If a comment never lands in the repository, it was theater.

How I will apply this process to future projects

Before the first feature branch I will freeze four things: the user who is in a hurry, the action the system is forbidden to take, the single process that must boot without secrets, and the health URL a host can probe. I will prototype screens cheaply, then move into an official scaffold, with the hosting contract in the tree on day one. I will finish GitHub and AWS identity on the machine that deploys, without pasting keys into chat. I will keep retrieval-locked knowledge wherever a hallucination can hurt a person or a payroll. Missing cloud evidence will be a punch list, not a screenshot I invent.

For these two apps the next move is that same process: complete an AWS session, create the GitHub connection, run create-service from the JSON already in the plant repo, and capture

both services Running in one console. Until then the portfolio I can stand behind is two public GitHub repositories and a plant desk that actually runs locally. That is a smaller claim than “fully deployed.” It is also the one I grew enough to make.